
DELIVERY TRACKING APP - CLIENT GUIDE

DriverTrack

An offline-first app for drivers to confirm deliveries on the spot, even with no internet. Real app screenshots inside - plus how it works in React Native: offline queueing, auto-sync and conflicts.

React Native - TypeScript - September 2026

Table of Contents

Part I - Using the App	3
1. What is DriverTrack?	3
2. Getting Started	3
3. App Walkthrough - With Screenshots	4
4. A Tour of the Screens	8
5. Understanding the Status Badges	9
6. Beyond the Task: Dark Mode, Arabic (RTL) and Splash	10
7. Demo Script: Eight Scenarios in Ten Minutes	10
8. Frequently Asked Questions	11
Part II - How It Works (Technical Overview)	12
9. Technology Stack	12
10. React Native in a Nutshell	12
11. System Architecture	13
12. The Outbox Pattern: How Offline Works	13
13. Sync States and Rules	14
14. Duplicate Protection (Idempotency)	14
15. Conflicts: When the Server Changed First	15
16. Data Model	15
17. Quality Assurance and Testing	16
18. Repository Workflow	16

19. Assumptions and Notes

16

Part I - Using the App

This first part is written for anyone who wants to install, run and use DriverTrack: product owners, reviewers and drivers. Part II explains how the app is built, for technical readers. All screenshots in this guide were captured from the real running app (Android emulator, 1344x2992).

1. What is DriverTrack?

DriverTrack is a mobile application for delivery drivers. Every morning a driver opens the app and sees the deliveries assigned to them for the day - in this build, an Egyptian sample route across Cairo, Giza and Alexandria, with amounts in Egyptian pounds (EGP). At each stop the driver opens the delivery, calls or navigates to the customer if needed, and then records the outcome: **Delivered** (required recipient name, optional note and optional photo proof) or **Failed** (required reason, optional note).

The defining feature of DriverTrack is that it **keeps working when the internet does not**. If a confirmation is recorded while offline, the app saves it safely on the device, clearly labels it as *Waiting to sync*, and sends it automatically as soon as connectivity returns. Nothing the driver records is ever lost - even if the phone is switched off or the app is closed and reopened.

Offline-first

works with no signal

24 / 24

automated tests passing

15 PRs

reviewed & merged work

AR + EN

Arabic and English, RTL

The app also protects against the classic problems of mobile data entry: it never submits the same confirmation twice (every action carries a unique idempotency key), and if someone else changes a delivery on the server before the driver's update arrives, the app detects the conflict and asks how to resolve it instead of silently overwriting data.

2. Getting Started

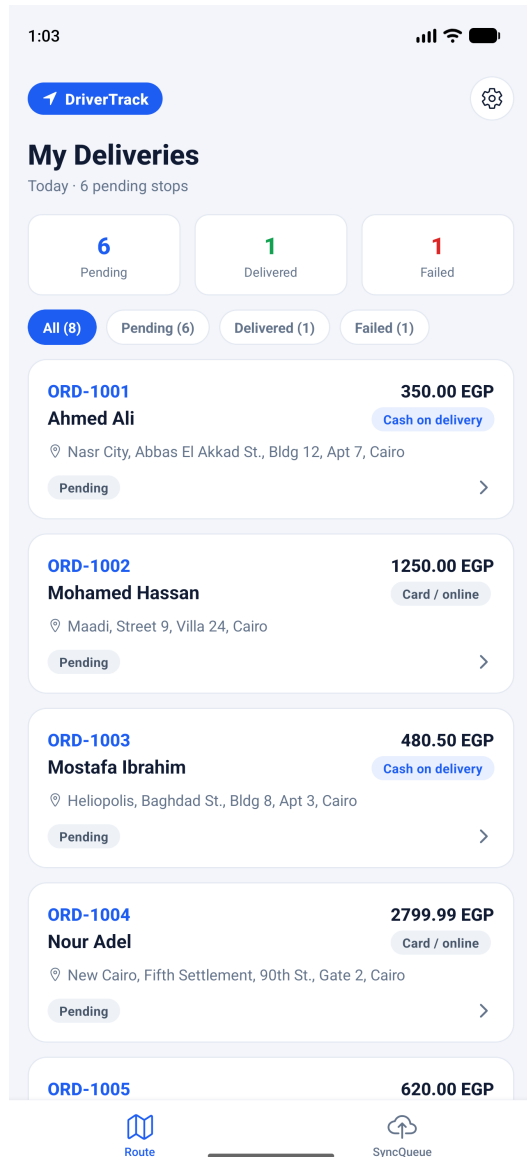
Step	Command	What it does
1	<code>npm install</code>	Installs the JavaScript dependencies (once).
2	<code>npm start</code>	Starts Metro, the development server. Keep this terminal open.
3	<code>npm run android</code>	Builds and installs the app on a connected device or emulator. First build: 3-10 minutes.
4	<code>npm test</code>	Runs the 24 automated tests. No device needed.

Table 1 - Installation and run commands

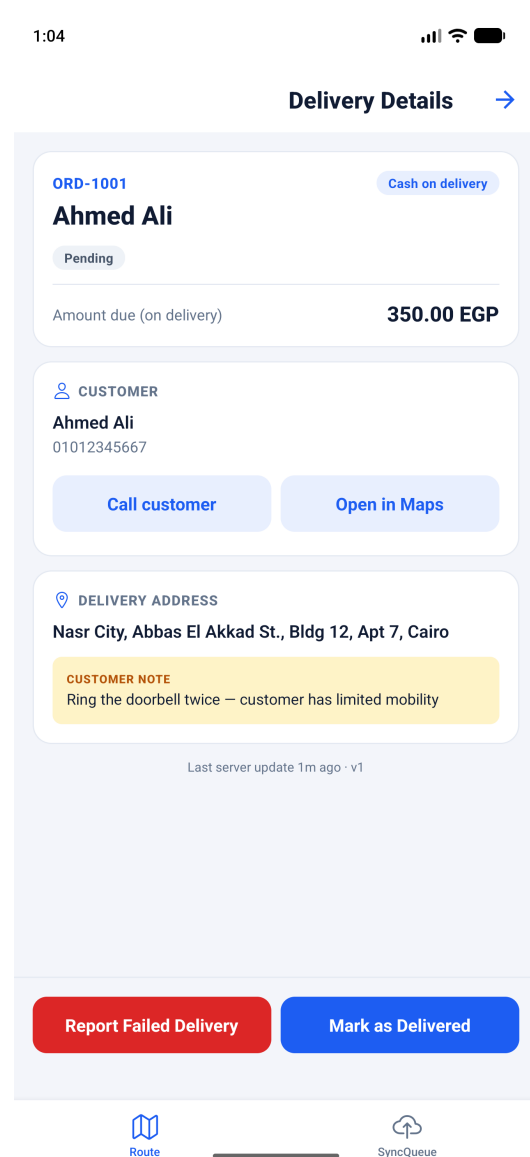
A ready-to-install **release APK** is also provided alongside this guide (see the GitHub release): copy it to any Android 8+ phone and install it - no development tools required.

3. App Walkthrough - With Screenshots

The fastest way to understand the app is to look at it. Each pair of screens below was captured from the running app while performing the daily workflow of a driver.



My Deliveries - the route home screen: today's stats, status filters, and every stop as a card with amount, payment method and status chips.



Delivery details: recipient, cash amount, one-tap call and maps, customer notes, and the two action buttons.

1:06

Mark as Delivered →

ORD-1001
Ahmed Ali
Nasr City, Abbas El Akkad St., Bldg 12, Apt 7, Cairo
350.00 EGP

RECIPIENT NAME *

Ahmed Ali

Required by the platform for proof of delivery.

DELIVERY NOTE

Optional — e.g. left with reception, gate code used...

0/240

PHOTO PROOF Optional



Attach photo proof

A photo of the delivered package strongly reduces disputes.

Confirm Delivery

Saved instantly and synced to the server.

Route SyncQueue

Marking a delivery as Delivered: recipient name required (pre-filled), optional note (240 chars) and optional photo proof.

1:06

Report Failed Delivery →

ORD-1001
Ahmed Ali
Nasr City, Abbas El Akkad St., Bldg 12, Apt 7, Cairo

FAILURE REASON *

☐ Customer unavailable

☐ Wrong / missing address

☐ Customer refused delivery

☐ Payment could not be collected

☐ Other reason

NOTE

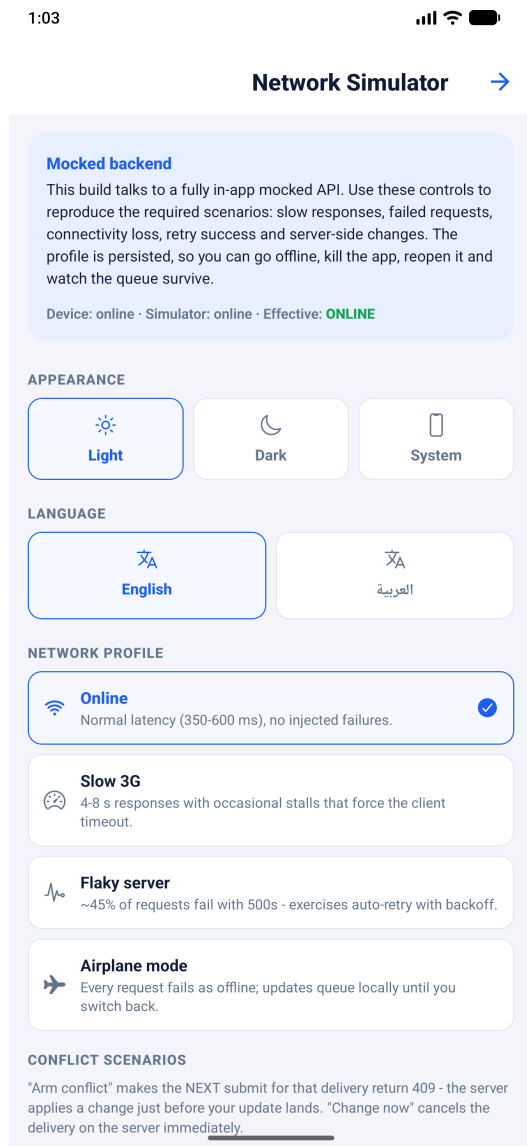
Optional — e.g. called twice, no answer...

Report Failed Delivery

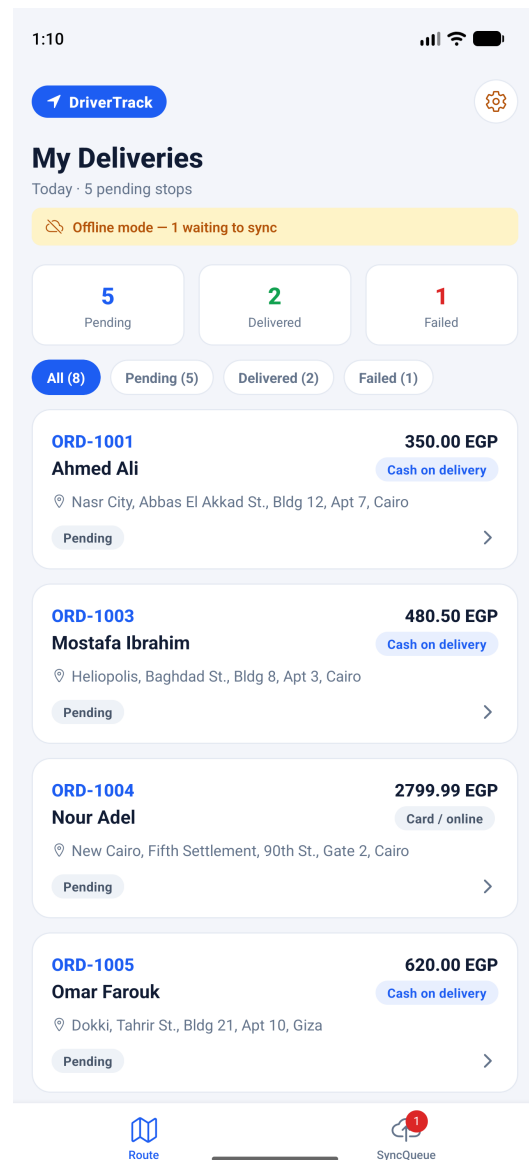
Saved instantly and synced to the server.

Route SyncQueue

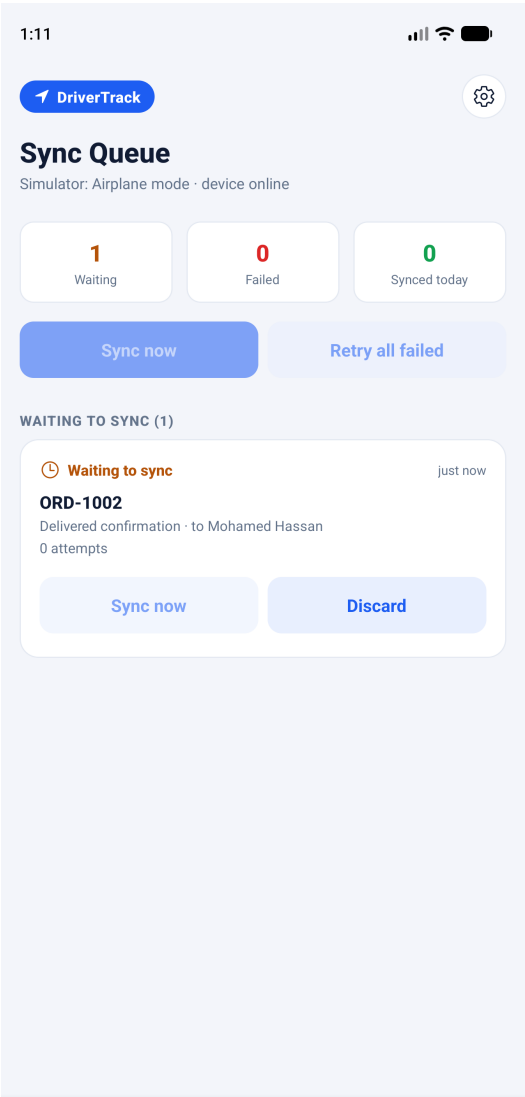
Reporting a failed delivery: a standardised reason must be selected, plus an optional note.



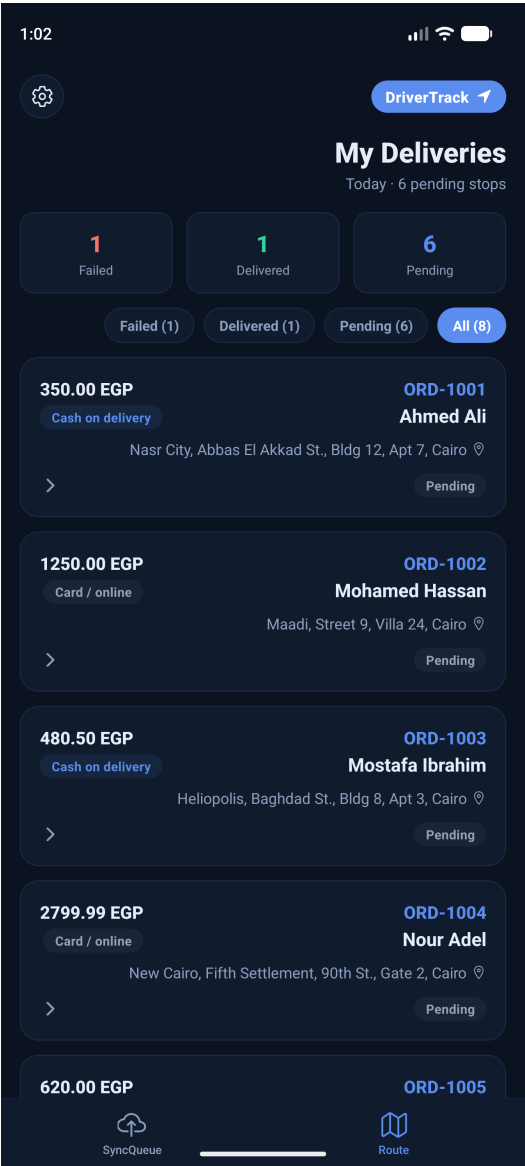
The Network Simulator (gear icon): switch between online, slow 3G, flaky server and airplane mode; choose appearance and language; arm a server-side conflict.



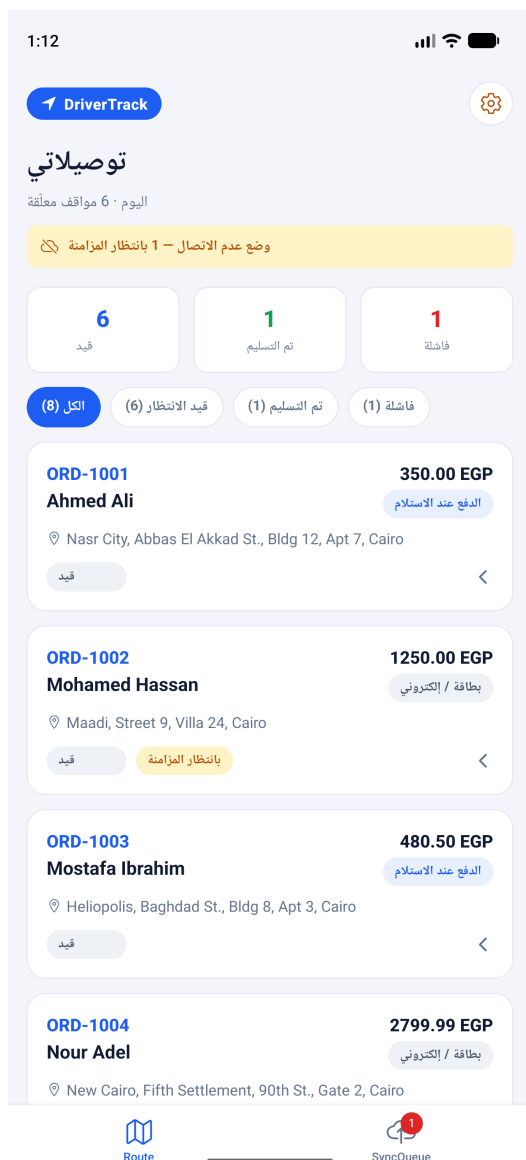
Offline mode: the confirmation was recorded with no signal - the amber banner counts it, the delivery shows Waiting to sync, and the queue badge appears.



The Sync Queue: stats at the top, the queued confirmation with its age and attempts, and Sync now / Discard actions.



Dark mode: the entire app is themable - light, dark, or following the system, selected in the settings screen.



Arabic with full RTL mirroring: switch language in settings - the whole layout, chips and banners mirror and translate instantly.



The branded app icon - consistent with the splash screen and the in-app header.

4. A Tour of the Screens

The app has two bottom tabs - **Route** and **Sync Queue** - plus detail and form screens that open on top. A gear icon in the header opens the Network Simulator.

Screen	What you see / what you do
My Deliveries (Route tab)	Stat cards (pending / delivered / failed), filter chips (All, Pending, Delivered, Failed), and one card per delivery: order number, customer, address, EGP amount, payment method, status + sync chips. Pull down to refresh. An amber banner appears while offline.
Delivery details	Everything needed at the stop: status chips, amount due, customer with one-tap Call, address with Open in Maps, customer notes, and (for resolved deliveries) the recorded recipient, failure reason or delivered-to name.

Screen	What you see / what you do
Mark as Delivered	Recipient name (required, pre-filled), optional note with 240-character counter, optional photo proof from camera or gallery.
Report Failed Delivery	Five standardised reasons (customer unavailable, wrong address, refused delivery, payment could not be collected, other) plus optional note.
Sync Queue tab	Stats (waiting / failed / synced today), needs-attention and waiting sections with per-item age, attempts, error and Retry / Discard / Resolve buttons, recently-synced list, and a queue badge on the tab icon.
Network Simulator	Drives the mocked backend: network profiles, appearance (light/dark/system), language (English/Arabic), conflict arming, server-side change, and mock data reset. Settings persist across restarts.

Table 2 - Screens and their purpose

5. Understanding the Status Badges

Every delivery carries two independent statuses: the **delivery status** (the outcome) and the **sync status** (whether the record has reached the server).

Delivery status	Meaning
Pending	Still to be delivered. Actions are available.
Delivered	Handed over successfully; recipient name recorded.
Failed	Could not be completed; reason recorded.
Cancelled	Cancelled on the server side (for example by customer support).

Table 3 - Delivery statuses

Sync status	Meaning	What the app does
Synced	Server confirmed the update.	Nothing - it is done.
Waiting to sync	Saved on the device; not yet at the server (usually offline).	Sends automatically on reconnect; retries in the background with growing delays.
Syncing	Being sent right now.	Brief progress state.
Failed to sync	3 attempts failed (or a permanent error).	Stops automatic retries; offers Retry now or Discard.
Conflict	The delivery changed on the server first (409).	Dialog: Discard my update, or Keep for dispatch review.

Table 4 - Sync statuses

Rule of thumb: a driver can always keep working. Any badge other than Synced simply means "the app has not finished talking to the server yet" - the record itself is already safe on the phone.

6. Beyond the Task: Dark Mode, Arabic (RTL) and Splash

Three product features were added on top of the core task, each merged as its own reviewed pull request.

Feature	What it does	Where
Dark mode	Full light and dark palettes; Light / Dark / System (follow the OS) selectable in settings and persisted.	Screenshot pair on the previous pages; settings screen.
Arabic + RTL	Complete Arabic translation of every screen, banner and dialog. Switching language mirrors the entire layout right-to-left (applies after a restart, as Android requires).	Language picker in settings; screenshot above.
Splash screen + icon	Branded launch screen (blue, white disc, navigation mark) on Android and iOS, plus the matching app icon in every required size, including Android adaptive icons.	Visible on every cold start.

Table 5 - Features beyond the task brief

7. Demo Script: Eight Scenarios in Ten Minutes

Open the Network Simulator with the gear icon whenever a step says so.

1. **Normal delivery.** Open any pending delivery, tap Mark as Delivered, confirm. The badge goes Syncing then Synced.
2. **Offline queueing.** Simulator: Airplane mode. Deliver another stop. It is accepted instantly and shows Waiting to sync, with an amber banner on the list.
3. **Survives a restart.** Still offline, close and reopen the app. The queued update is still in the Sync Queue - nothing was lost.
4. **Automatic sync.** Simulator: back to Online. The queue drains by itself and the update turns Synced.
5. **Failing server, automatic retry.** Simulator: Flaky server. Watch attempts climb on the Sync Queue, then succeed. After 3 failed attempts an action parks as Failed to sync.
6. **Manual retry.** Tap Retry now on any failed item (or Retry all failed).
7. **Slow network.** Simulator: Slow 3G. Deliveries still succeed; occasionally a request stalls, the app times out and retries by itself.
8. **Conflict.** Simulator: Arm conflict on a pending delivery, then deliver it. The server changes the order first and answers 409; the app shows the resolution dialog (Discard my update / Keep for dispatch review).

8. Frequently Asked Questions

Q: I recorded a delivery with no signal. Is it saved?

A: Yes. It is written to the phone's persistent storage the moment you confirm, before any network is attempted.

Q: I closed the app / the phone died. Is my update gone?

A: No. Queued updates are persisted and reloaded on the next launch.

Q: When does the app retry by itself?

A: When connectivity returns, roughly every 20 seconds while open (with growing delay per failed attempt), and at startup if updates are queued. After 3 failed attempts it waits for a manual retry.

Q: Can a delivery be confirmed twice by accident?

A: No - one pending update per delivery, duplicate submits blocked in the UI, and the server recognises repeated submissions of the same confirmation.

Q: What is a conflict, in plain words?

A: Someone changed the delivery on the server (for example the customer cancelled through support) before the driver's confirmation arrived. The app asks what to do instead of overwriting silently.

Q: Is Arabic fully supported?

A: Yes - every screen, chip, banner and dialog is translated, and the layout mirrors right-to-left. The direction change applies after an app restart, as Android requires.

Part II - How It Works (Technical Overview)

How DriverTrack is built with React Native, how the offline synchronisation works, and where every behaviour lives in the codebase.

9. Technology Stack

React Native 0.87 (bare CLI) with TypeScript. Each dependency keeps the core logic testable without a device.

Technology	Role in the app	Why this choice
React Native 0.87 + TypeScript	Cross-platform UI from one JavaScript codebase rendering native views.	Industry standard, fast iteration, compile-time type safety.
React Navigation v7	Root stack + bottom tabs (Route / Sync Queue) + native stack screens.	Standard navigation library.
Zustand	App-wide state store.	Tiny API, works outside React, trivially testable.
AsyncStorage	Persistent key-value storage.	Reliable persistence for the cache, outbox and settings.
NetInfo	Device connectivity events.	Official community library; drives auto-sync on reconnect.
react-native-image-picker	Photo proof attachment.	Mature camera + gallery picker.
Jest	Unit testing.	Standard runner; core tests run in plain Node.
MockServer (in-app)	Fake backend with controllable behaviour.	Reproduces every required scenario deterministically.

Table 6 - Libraries and their roles

10. React Native in a Nutshell

React Native lets one JavaScript codebase drive real native interfaces. Developers write **components** - functions that describe the UI as a function of their data - and React Native maps them onto actual native views. The result behaves like a native app, not a web page.

Three mechanisms matter here. **State**: the UI is a projection of state, so all screens read from one store (Zustand) and re-render when it changes. **Metro**: the bundler serving JavaScript to the app, with Fast Refresh showing changes in about a second. **Native modules**: platform features - storage, connectivity, image picking, the RTL flags - come as small native libraries behind JavaScript APIs.

The consequence for DriverTrack: all business logic - queue, retry rules, conflicts - lives in plain TypeScript modules with no React and no native dependencies, which is why the whole sync lifecycle is tested in seconds

with Jest. The visual layer (theme, RTL, navigation) is pure React Native on top.

11. System Architecture

Layered so each layer only talks to its neighbour, and the sync engine runs - and is tested - without any UI.

Layer	Location	Responsibility
Screens (UI)	src/screens, src/components	Render state via selectors; call store actions. No fetching, no storage access.
App store	src/store/useAppStore.ts	Single source of truth: deliveries, outbox, sync log, settings. Persists through repositories and delegates network work to the engine.
Repositories	src/storage	Typed persistence over AsyncStorage: delivery cache, outbox queue, synced log, simulator, settings.
Sync engine	src/sync/syncEngine.ts	Drains the outbox FIFO with retry/backoff; detects conflicts; emits events.
Connectivity	src/sync/connectivity.ts	Wraps NetInfo into one boolean stream.
Mock API	src/api	Implements the Api interface: latency profiles, injected failures, offline mode, idempotency registry, 409 conflicts.

Table 7 - Layers and responsibilities

Data flows one way: **Screens** -> **Store** -> **SyncEngine** -> **Mock API**, with results returning as events the store applies and persists. The engine receives its dependencies from outside, so tests substitute in-memory doubles - 34 tests covering the whole lifecycle run in about three seconds. Swapping the mock for a real backend means implementing one interface (src/api/types.ts); nothing else changes.

12. The Outbox Pattern: How Offline Works

DriverTrack uses the classic **outbox pattern**, the same approach used by banking and logistics apps that must never lose a transaction.

1. **Record.** The confirmation is created with a fresh unique id (the `client_action_id`) and written to the outbox in persistent storage - before any network call. A crash one millisecond later would still leave it on disk.
2. **Apply locally.** The cached delivery is optimistically updated so the UI reflects the driver's intent immediately, marked Waiting to sync.
3. **Hand off.** The engine syncs if online; otherwise the action waits.
4. **Send.** When online, queued actions are sent oldest-first; the photo proof (if any) uploads once before the confirmation.

5. **Confirm or retry.** Success removes the action and stores the server's state. A retryable failure (no network, timeout, server error) increments attempts and re-queues with a longer delay - 5s, 10s, 20s, capped at 60s.

6. **Park or escalate.** After 3 failed attempts the action parks as Failed to sync for manual retry. A 409 conflict parks separately with the server state for the resolution dialog.

13. Sync States and Rules

State	Meaning	Moves on
waiting	Saved, not yet accepted by the server.	Engine picks it up when online (submit, reconnect, 20s timer respecting backoff).
syncing	A request is in flight.	Success -> synced (removed); failure -> waiting / failed / conflict.
failed	Exhausted automatic attempts or permanent error.	Manual retry resets attempts.
conflict	Server returned 409 with its own state.	User decision: Discard, or Keep for dispatch review.
synced	Server accepted it.	Terminal; shown in Recently synced.

Table 8 - The sync state machine

14. Duplicate Protection (Idempotency)

A slow network can deliver a request but lose the acknowledgement - the client retries, and without protection the customer would be marked delivered twice. Three independent layers prevent this:

Layer	Protection
UI	One pending action per delivery; while it exists, the action buttons are replaced by the sync status card.
Client engine	All attempts are serialised and reuse the same <code>client_action_id</code> .
Server	A registry of processed ids: a repeat submission returns the original result marked duplicate and does not mutate state.

Table 9 - Three layers of duplicate protection

15. Conflicts: When the Server Changed First

Between the tap on confirm and the arrival of the request, the delivery may have changed on the server - the classic case is a customer cancelling through support. The mock server rejects such a submit with HTTP 409 and includes its current state. The engine parks the action as **conflict** - never auto-retried - and the app explains what happened: **Discard my update** removes it and restores the pending view; **Keep for dispatch review** leaves it parked in the queue under Needs attention.

16. Data Model

Field names mirror the task's suggested API contract.

Delivery field	Type	Notes
id / order_number	number / string	Server identity.
customer_name / phone	string	Shown on cards; one-tap call.
address	string	Opens in Google Maps.
amount_due	number	EGP, two decimals.
payment_method	cash card	Rendered as a chip.
status	pending delivered failed cancelled	Server-owned outcome.
recipient_name / note / failure_reason	string null	Filled by the confirmation or failure flow.
version / updated_at	number / timestamp	Bumped on every server mutation; used in conflicts.

Table 10 - Delivery

PendingAction field	Type	Notes
id = client_action_id	UUID string	Idempotency key for every attempt.
delivery_id	number	One action per delivery.
type	complete fail	Which confirmation.
status	waiting syncing failed conflict	State machine.
payload	complete or fail payload	Exactly what the API expects.
attempts / last_attempt_at	number / timestamp	Backoff scheduling.
photo_uri / proof_uploaded	local uri / boolean	Proof uploaded once before the complete call.
server_delivery	Delivery null	Attached to conflicts.

Table 11 - PendingAction (the outbox entry)

17. Quality Assurance and Testing

24 unit tests lock down the important behaviours and run in plain Node in about three seconds, plus 10 more covering the i18n dictionaries and theme palettes (34 in total).

Area	What the tests prove
Mock server	Idempotency: the same client_action_id never applies twice. Conflicts return 409 with server state. Offline fails then succeeds after reconnect. Proof attaches.
Persistence	Queued actions and the cache survive app restarts. Corrupt storage falls back to defaults.
Sync engine	Success, offline no-op, flush on reconnect, retryable failures with backoff, parking as failed, conflicts never auto-retried, proof once, backoff maths.
i18n + theme	Arabic defines exactly the English keys; interpolation; RTL flag; palettes complete and consistent.

Table 12 - Test coverage

18. Repository Workflow

The history mirrors a professional delivery: fifteen pull requests, each documented and merged in order - scaffold, mock API, persistence, sync engine, screens, forms, sync queue + simulator, tests, docs, client guide, dark mode, Arabic/RTL, splash screen, Egyptian dataset, app icon, and this navigation hardening.

Folder	Contents
src/api	Mock server, network profiles, Api interface, errors.
src/storage	KeyValue port + repos (cache, outbox, synced log, simulator, settings).
src/sync, src/store	Connectivity, SyncEngine, Zustand store.
src/screens, src/components	Six screens and reusable themed UI.
src/i18n, src/theme	Arabic/English dictionaries + provider; light/dark palettes.
tests	34 Jest unit tests.
docs	This guide, ARCHITECTURE.md, app screenshots.

Table 13 - Where things live

19. Assumptions and Notes

The significant assumptions, also listed in the repository README:

- **Conflict definition:** a submit conflicts when the delivery is no longer pending on the server; the response carries the authoritative state.
- **Keep for dispatch review** leaves the conflicting action parked; it never auto-retries.

- **Failed syncs need a manual retry** after 3 automatic attempts, so failure stays visible.
- **Photo proof** is optional, uploaded once before the confirmation.
- **Single driver, no authentication** - the task focuses on delivery and sync behaviour.
- **Egyptian sample data:** amounts in EGP with two decimals; addresses across Cairo, Giza and Alexandria; local 01x mobile numbers.
- **RTL direction** applies after an app restart when switching language, as required by Android's layout mirroring.

Next step for production: replace the MockServer with a real HTTP implementation of the same Api interface (one file), keep everything else - the outbox, retries, conflicts and all tests - exactly as they are.